

Desarrollo de un *pipeline* de aprendizaje continuo para chatbots basados en PLN

Ing. Porra Bustos, Matias Exequiel (UTN-FRLR)

Carrera de Especialización en Inteligencia Artificial

Director: Dr. Ing. Cárdenas Rodrigo (FIUBA)

Jurados:

Esp. Ing. Torrent Leandro (FIUBA)

Esp. Ing. Freyer Nicolás (FIUBA)

Jurado 3 (pertenencia)

Ciudad Autónoma de Buenos Aires, mayo de 2025

Resumen

El presente trabajo desarrolla una solución avanzada para la automatización de la comunicación empresarial, orientada a emprendedores y pequeñas empresas. El sistema desarrollado consiste en un chatbot que integra técnicas de procesamiento del lenguaje natural (PLN) y gestión de bases de datos, articulado en un *pipeline* de aprendizaje continuo. Este chatbot está diseñado para optimizar la interacción con los clientes, ya que genera respuestas precisas y relevantes a sus consultas en lenguaje natural. Además, el sistema se retroalimenta de las interacciones previas para mejorar su rendimiento de manera constante y adaptarse mejor a las necesidades de los usuarios.

Agradecimientos

A mi novia por apoyo incondicional durante todo el proceso de desarrollo de este trabajo.

A mi familia, por estar siempre presente.

A mi tutor, por su guía y su conocimiento.

A Michael Schreiber por sus grandes aportes y guía en mi formación profesional en IA.

Índice general

Resumen	I
1. Introducción general	1
1.1. Motivación	1
1.1.1. Definición de chatbot	1
1.1.2. Chatbots como solución eficiente	1
1.2. Alcance y objetivos	2
1.2.1. Objetivos principales	2
1.2.2. Alcance del trabajo	3
1.3. Estado del arte	3
1.3.1. Recursos para chatbots	3
Servicios basados en APIs	3
Modelos locales	4
1.3.2. Tendencias actuales	4
1.3.3. Consideraciones éticas en el uso de chatbots	4
Privacidad y manejo de datos	4
Limitaciones temáticas y control de contenido	4
Transparencia y límites de automatización	5
Equidad y sesgos	5
2. Introducción específica	7
2.1. Requerimientos	7
2.2. Modelos de inteligencia artificial para PLN	8
2.2.1. Modelos de Lenguaje modernos	9
2.3. Modelos de chatbots: RAG vs <i>Fine-Tuning</i>	9
2.3.1. <i>Retrieval Augmented Generation</i> (RAG)	9
2.3.2. <i>Fine-Tuning</i>	10
2.3.3. Ventajas y desventajas de los modelos RAG y <i>Fine-Tuning</i>	11
3. Diseño e implementación	13
3.1. Diseño de la arquitectura	13
3.1.1. Propuestas de implementación	13
3.1.2. Problemas y mitigaciones	19
3.1.3. Elección de la arquitectura de chatbot	20
3.2. Arquitectura final propuesta	20
3.2.1. Tecnologías suplementarias	22
3.2.2. Funcionamiento general	22
3.3. Implementación	24
3.3.1. Preparación de los datos de contexto	24
3.3.2. Métricas, clasificadores y análisis semántico en sistemas conversacionales	25

3.3.3.	Sistema de evaluación automatizada y mejora continua sin reentrenamiento	26
3.3.4.	Despliegue del sistema	27
4.	Ensayos y resultados	29
4.1.	Evaluación de calidad en sistemas conversacionales	29
4.1.1.	Criterios y mecanismos de evaluación	29
4.1.2.	Evaluación, observabilidad y trazabilidad	29
4.2.	Despliegue de la interfaz de pruebas	30
4.2.1.	Interfaz de usuario	30
	Chatbot	30
	Métricas	31
	Administración de contextos	32
4.3.	Evaluación de escenarios del chatbot	33
4.3.1.	Escenario 1: interacción básica	33
4.3.2.	Escenario 2: consultas con múltiples pasos	33
4.3.3.	Escenario 3: consultas fuera de contexto	33
4.4.	Resultados de las pruebas	34
4.4.1.	Pruebas para la versión 0	34
	Pruebas de interacción básica	34
	Pruebas de consultas con múltiples pasos	34
	Pruebas de consultas fuera de contexto	34
	Conclusiones de la versión 0	35
4.4.2.	Pruebas para la versión 1	35
	Pruebas de interacción básica	35
	Pruebas de consultas con múltiples pasos	35
	Pruebas de consultas fuera de contexto	35
	Conclusiones de la versión 1	36
4.4.3.	Pruebas para la versión 2	36
	Pruebas de interacción básica	36
	Pruebas de consultas con múltiples pasos	36
	Pruebas de consultas fuera de contexto	37
	Conclusiones de la versión 2	37
4.4.4.	Pruebas para la versión 3	37
	Pruebas de interacción básica	37
	Pruebas de consultas con múltiples pasos	37
	Pruebas de consultas fuera de contexto	37
	Conclusiones de la versión 3	38
4.4.5.	Comparación de versiones del chatbot según métricas de aplicación	38
5.	Conclusiones	41
5.1.	Conclusiones generales	41
5.1.1.	Cumplimiento de objetivos y planificación	41
5.1.2.	Gestión de riesgos y adaptaciones	41
5.1.3.	Resultados, aprendizajes y aportes	41
5.2.	Próximos pasos	42
5.3.	Síntesis final	42
	Bibliografía	43

Índice de figuras

1.1. Relación entre los objetivos centrales.	2
2.1. Diagrama de flujo de una consulta en un chatbot con arquitectura RAG.	10
3.1. Diagrama general del sistema para la versión 1.	15
3.2. Diagrama general del sistema para la versión 2.	16
3.3. Diagrama general del sistema de métricas de la versión 2.	17
3.4. Ejemplo de redefinición de pregunta.	18
3.5. Diagrama general del flujo del <i>pipeline</i> del sistema.	22
3.6. Diagrama de flujo de funcionamiento.	23
3.7. Ejemplo de formato de contexto.	24
3.8. Diagrama de flujo de observación de métricas.	27
4.1. Frontend del chatbot.	31
4.2. Métricas por categoría temática.	31
4.3. Ejemplo del frontend con métricas de preguntas bien calificadas. . .	32
4.4. Ejemplo del frontend con métricas de preguntas mal calificadas. . .	32
4.5. Vista del frontend para la administración de contextos.	33

Índice de tablas

2.1. Ventajas y desventajas de los modelos RAG y <i>Fine-Tuning</i>	11
4.1. Comparación de versiones del chatbot	38

Capítulo 1

Introducción general

En este capítulo se presentan las motivaciones que impulsaron el desarrollo del sistema, diseñado para automatizar la comunicación entre emprendedores y sus clientes. Se ofrece, además, una explicación detallada sobre qué son los chatbots y las razones por las que su uso resulta fundamental en este contexto. Finalmente, se describen los objetivos principales del trabajo, centrados en la facilidad de implementación y en la mejora de la eficiencia operativa, junto con el alcance definido durante la planificación.

1.1. Motivación

El sistema desarrollado en el presente trabajo surge de la necesidad de proporcionar a emprendedores una herramienta que les permita automatizar la comunicación con sus clientes de manera eficiente y personalizada. Esta herramienta les ofrece una ventaja competitiva al optimizar la gestión de sus interacciones con los usuarios y les permite mejorar la eficiencia operativa, además de reducir los costos asociados a la atención al cliente.

1.1.1. Definición de chatbot

Un chatbot es un programa de software que emplea técnicas de Procesamiento del Lenguaje Natural (PLN) [1], para interpretar y contestar automáticamente las consultas de los usuarios de manera coherente y eficiente. Esto facilita la automatización de interacciones comunes y mejora la experiencia del cliente.

1.1.2. Chatbots como solución eficiente

A continuación, se destacan algunas de las razones principales por las que un chatbot es la solución ideal para este trabajo:

- Disponibilidad permanente: los chatbots están disponibles en todo momento, lo que les permite a las empresas atender consultas de sus clientes a cualquier hora del día.
- Reducción de costos: al automatizar la atención al cliente, los chatbots ayudan a las empresas a reducir los costos asociados al personal humano sin comprometer la calidad del servicio.
- Recolección de datos valiosos: los chatbots pueden recopilar y analizar información relevante sobre las interacciones con los clientes, lo que les permite a las empresas ajustar sus estrategias de manera eficiente.

- Optimización de tiempos de respuesta: la capacidad de generar respuestas inmediatas en función de consultas predefinidas optimiza los tiempos de respuesta.

1.2. Alcance y objetivos

1.2.1. Objetivos principales

El trabajo se enfoca en tres objetivos principales que se interrelacionan para ofrecer una solución integral a pequeños emprendedores. Cada uno de estos objetivos está orientado a asegurar que la herramienta sea accesible, fácilmente implementable y de bajo costo, para garantizar una experiencia eficiente y optimizada para las empresas que la adopten. Como se observa en la figura 1.1, estos tres ejes convergen para conformar un enfoque cohesivo orientado a la satisfacción del cliente.

A continuación, se detallan los objetivos principales:

- Proporcionar acceso a pequeños emprendedores: desarrollar un sistema que sea de fácil adopción y pueda ser utilizado por pequeñas empresas, independientemente de sus conocimientos técnicos.
- Garantizar una fácil implementación: diseñar una solución replicable, de modo que cualquier empresa pueda contar con el chatbot simplemente al proporcionar el contexto necesario para su entrenamiento. No será necesario disponer de infraestructura compleja ni personal especializado.
- Reducir el costo de mantenimiento: proporcionar una herramienta con un bajo costo de mantenimiento, tanto en términos económicos como de tiempo, para que los emprendedores puedan centrarse en su negocio principal.

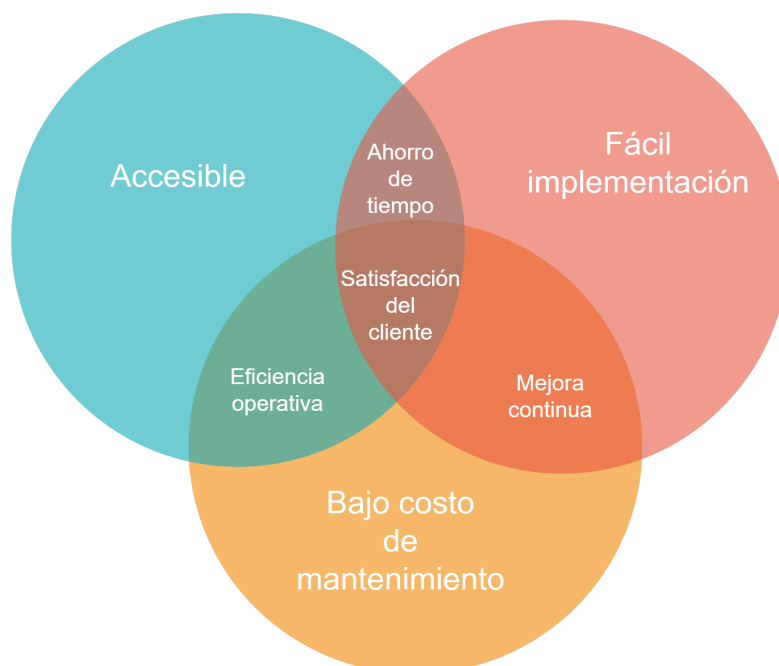


FIGURA 1.1. Relación entre los objetivos centrales.

1.2.2. Alcance del trabajo

Durante la planificación del trabajo se propusieron las siguientes actividades:

- Diseño, desarrollo e implementación de un *pipeline* completo para la creación y gestión de chatbots basados en PLN, con capacidad de aprendizaje continuo.
- Desarrollo de algoritmos para el preprocesamiento de información y generación de respuestas relevantes y coherentes.
- Establecimiento de métricas y procedimientos de evaluación para medir la calidad y eficacia de las respuestas del chatbot.
- Implementación de un sistema de retroalimentación que utilice los datos de interacciones con usuarios para mejorar el rendimiento del modelo.

El presente trabajo excluye:

- Desarrollo de interfaces de usuario avanzadas, por fuera de las básicas necesarias para probar el *pipeline*.
- Integración con sistemas externos, como bases de datos o plataformas de gestión de clientes.
- Implementación del sistema en un entorno productivo.
- Actividades de marketing o promoción del producto final.

1.3. Estado del arte

Los chatbots se han integrado en numerosos aspectos de la vida diaria y en diversos canales de comunicación. Se encuentran presentes en redes sociales como Facebook e Instagram, donde facilitan la interacción con empresas. Además, muchas organizaciones y entidades gubernamentales, han implementado chatbots para mejorar la atención al ciudadano. Plataformas como IBM Watson, Google Dialogflow y Microsoft Bot Framework permiten a las empresas crear chatbots personalizados con gran flexibilidad.

1.3.1. Recursos para chatbots

Servicios basados en APIs

Los servicios basados en API (por sus siglas en inglés, *Application Programming Interface*) simplifican la implementación de chatbots al proporcionar interfaces estandarizadas para el acceso a funcionalidades avanzadas. Estos servicios permiten una mayor personalización y flexibilidad en el diseño de chatbots, que facilitan la integración con otros sistemas y plataformas. La utilización de APIs permite a las empresas adaptar las soluciones a sus necesidades específicas sin necesidad de desarrollar toda la infraestructura desde cero.

Modelos locales

Los modelos locales como Llama 3.1, Phi 3, Gemma y Mistral son soluciones robustas que operan sin depender de servicios externos. Cada uno de estos ofrece características y ventajas específicas, como mayor control sobre los datos y la posibilidad de operar en entornos con restricciones de privacidad. Su uso es particularmente valioso cuando se requiere un alto nivel de personalización y seguridad en la gestión de datos.

1.3.2. Tendencias actuales

Cada vez más empresas optan por utilizar servicios basados en API, que han hecho más accesible y económico el despliegue de chatbots sofisticados. Estas herramientas simplifican la implementación y permiten la creación rápida de soluciones adaptadas a las necesidades específicas de cada empresa o entidad. Aunque los modelos locales son valiosos por su robustez y capacidad para operar sin depender de servicios externos, son útiles especialmente cuando se requiere un mayor control o confidencialidad.

1.3.3. Consideraciones éticas en el uso de chatbots

El desarrollo e implementación de chatbots inteligentes plantea no solo desafíos técnicos, sino también importantes cuestiones éticas, especialmente cuando se emplean en contextos sensibles como la atención al cliente, la gestión de datos o la interacción con usuarios vulnerables. Estas consideraciones resultan esenciales para garantizar que la automatización de las interacciones no comprometa la privacidad, la transparencia ni la equidad.

Privacidad y manejo de datos

Una de las principales preocupaciones éticas se relaciona con la protección de los datos personales. Al interactuar con usuarios, los chatbots pueden recolectar información sensible de manera no intencionada. Por ello, es fundamental que estos sistemas se diseñen con el objetivo de limitar el almacenamiento innecesario de datos y garantizar su anonimato.

Limitaciones temáticas y control de contenido

Otro aspecto relevante es el control sobre el tipo de consultas que el chatbot puede atender. En entornos públicos o semiabiertos, es posible que los usuarios formulen preguntas relacionadas con temas delicados, como política, religión o salud. La generación automatizada de respuestas en estos ámbitos puede derivar en desinformación o respuestas inapropiadas. Por este motivo, se incorporan mecanismos de filtrado, denominados *guardrails*, que permiten detectar y bloquear consultas fuera del dominio permitido, lo que contribuye a la seguridad y alineación ética del sistema.

Transparencia y límites de automatización

Es fundamental que los usuarios sean conscientes de que interactúan con un sistema automatizado y no con una persona. Esta transparencia evita la generación de falsas expectativas y reduce el riesgo de que se confíe en el chatbot para la toma de decisiones críticas. Asimismo, el sistema debe contar con mecanismos que le permitan reconocer sus propias limitaciones y, en caso de no disponer de información contextual suficiente, abstenerse de responder o derivar la consulta a un operador humano.

Equidad y sesgos

Los modelos de lenguaje empleados por los chatbots pueden reproducir sesgos presentes en los datos de entrenamiento. Si no se abordan de manera adecuada, dichos sesgos pueden perpetuar discriminaciones o estereotipos. Por ello, resulta esencial realizar un análisis exhaustivo de las fuentes de datos utilizadas en el entrenamiento, así como aplicar técnicas de mitigación. Entre estas se incluyen la revisión crítica de los conjuntos de datos, la identificación sistemática de sesgos y la implementación de estrategias orientadas a su corrección. Asimismo, es fundamental efectuar pruebas continuas que permitan verificar que el sistema se mantenga libre de sesgos a lo largo del tiempo.

Capítulo 2

Introducción específica

Este capítulo explora los requisitos fundamentales para el desarrollo de un sistema de chatbot, que abarcan aspectos funcionales, de documentación, pruebas, interfaz y rendimiento. Además, se examinan los distintos tipos de chatbots y modelos de inteligencia artificial aplicados al procesamiento de lenguaje natural, con énfasis en sus características y usos.

2.1. Requerimientos

En esta sección se presentan los requerimientos específicos de software del sistema. Estos fueron definidos al inicio del desarrollo del proyecto. El objetivo principal consiste en construir un chatbot inteligente, basado en la arquitectura RAG, optimizado para ofrecer respuestas contextualizadas, precisas y eficientes. Además, se requiere que el sistema disponga de una interfaz adaptable y garantice trazabilidad completa de las interacciones.

1. Requerimientos funcionales fueron:

- a) El sistema debe ser capaz de procesar y comprender mensajes de texto entrantes.
- b) El chatbot debe poder proporcionar respuestas relevantes y precisas a las consultas de los usuarios.
- c) Los usuarios deben tener la posibilidad de interactuar con el chatbot a través de una interfaz de usuario.
- d) Se requiere una interfaz de usuario que facilite el proceso de reentrenamiento del chatbot mediante el uso de los historiales de conversación.

2. Requerimientos de documentación:

- a) Documentar detalladamente las tecnologías utilizadas y sus principales características, así como las particularidades de diseño de cada una.
- b) El sistema no almacenará datos personales de los usuarios, sino que se limitará a responder preguntas frecuentes. En caso de requerirse almacenamiento de datos, se utilizará una plataforma con medidas de seguridad integradas, como autenticación mediante logins con Google.
- c) Entregar una memoria técnica que contenga información detallada sobre la implementación del sistema, que incluya aspectos técnicos, arquitectura y decisiones de diseño.

- d) Proporcionar un registro de avance que documente los hitos alcanzados durante el desarrollo del proyecto, que contemple fechas de cumplimiento y descripciones de las tareas realizadas.
- 3. Requerimientos de testing:
 - a) Se llevarán a cabo pruebas en diferentes escenarios y con diferentes tipos de contexto de datos, para garantizar la fiabilidad y precisión del chatbot.
- 4. Requerimientos de la interfaz:
 - a) Se requiere una interfaz interactiva, que permita hacer preguntas y obtener respuestas en tiempo real dentro del mismo entorno de interacción.
- 5. Requerimientos de rendimiento:
 - a) Se establece un límite máximo de tiempo de respuesta de 1 minuto, para asegurar una experiencia satisfactoria para el usuario.

2.2. Modelos de inteligencia artificial para PLN

El procesamiento de lenguaje natural (PLN) ha avanzado significativamente gracias al desarrollo de diversas técnicas de inteligencia artificial. Estas se pueden clasificar en varias categorías, cada una con su propio enfoque y aplicación.

Modelos basados en reglas: estos modelos utilizan gramáticas y diccionarios para analizar el lenguaje. Aunque son efectivos en tareas específicas, su rigidez y la necesidad de una extensa programación manual limitan su aplicabilidad en contextos más amplios.

Modelos estadísticos: a medida que los datos comenzaron a acumularse, los modelos estadísticos, como los *n*-gramas, se convirtieron en populares. Estos modelos predicen la probabilidad de una palabra en función de las anteriores. Sin embargo, su dependencia de datos limitados puede resultar en un contexto insuficiente, lo que afecta la calidad de las predicciones.

Modelos de aprendizaje profundo: con el auge del aprendizaje profundo, arquitecturas como RNN (Redes Neuronales Recurrentes), LSTM (por sus siglas en inglés, *Long Short-Term Memory*) y *transformer* han revolucionado el PLN. Estos modelos pueden captar patrones complejos en grandes volúmenes de datos, lo que les permite realizar tareas como la traducción automática y la generación de texto de manera más efectiva.

2.2.1. Modelos de Lenguaje modernos

Modelos como BERT (por sus siglas en inglés, *Bidirectional Encoder Representations from Transformers*) y GPT (por sus siglas en inglés, *Generative Pre-trained Transformer*) han transformado el campo del procesamiento de lenguaje natural (PLN), al ofrecer enfoques innovadores para comprender y generar texto.

- GPT: este modelo utiliza técnicas de aprendizaje profundo para generar texto coherente y contextualmente relevante. Su capacidad para crear respuestas naturales ha revolucionado la interacción de los chatbots con los usuarios, lo que permite conversaciones más fluidas y precisas.
- BERT: a diferencia de otros modelos, BERT se centra en el análisis bidireccional del texto, lo que le permite entender el contexto de manera más profunda. Esta característica mejora la identificación de intenciones y la respuesta a preguntas complejas, lo que ayuda a crear chatbots más competentes en diálogos matizados.

2.3. Modelos de chatbots: RAG vs Fine-Tuning

Los chatbots se pueden clasificar según sus arquitecturas y métodos de entrenamiento. Entre las más destacadas se encuentran *Retrieval Augmented Generation* (RAG) y *fine-tuning*. Ambas arquitecturas ofrecen ventajas complementarias, lo que las convierten en opciones populares en el desarrollo de chatbots efectivos [2].

2.3.1. Retrieval Augmented Generation (RAG)

RAG integra técnicas de generación de lenguaje natural con mecanismos de recuperación de información que operan sobre una base de conocimiento estructurada. Ante una consulta del usuario, el modelo emplea *embeddings* para localizar y extraer los segmentos más relevantes del contexto almacenado, con el fin de generar una respuesta coherente y fundamentada [3]. Este conocimiento relevante se inyecta en el *prompt* [4] enviado al modelo de lenguaje (LLM). El *prompt* incluye tanto la pregunta como el contexto relacionado, lo que permite a la LLM generar respuestas más precisas y contextualizadas. La incorporación de datos externos en este proceso enriquece el conocimiento del modelo en tiempo real y disminuye las posibilidades de alucinaciones respuestas que, aunque parecen plausibles, son incorrectas. Además, RAG utiliza bases de datos vectoriales y la búsqueda semántica para recuperar información relevante, lo que mejora la precisión y relevancia de las respuestas del chatbot. Este enfoque es especialmente útil en situaciones donde se requiere información actualizada o especializada, como en consultas de productos o en el soporte técnico. La figura 2.1 representa el flujo de información en una arquitectura RAG desde la consulta inicial hasta la generación final de la respuesta.

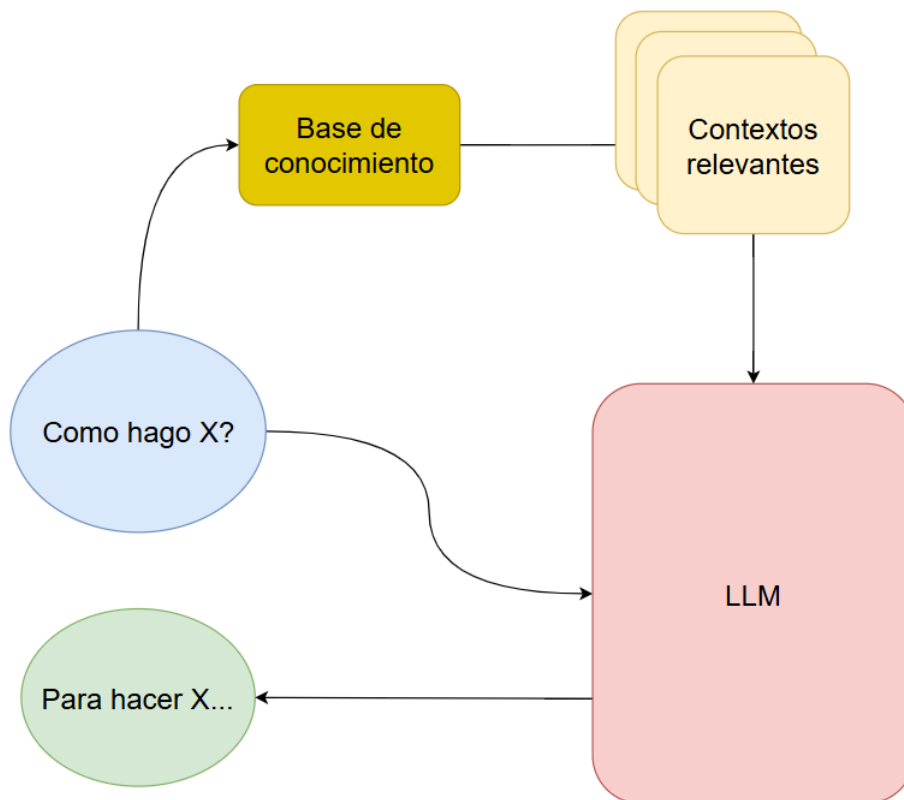


FIGURA 2.1. Diagrama de flujo de una consulta en un chatbot con arquitectura RAG.

2.3.2. *Fine-Tuning*

El *fine-tuning* implica ajustar un modelo de lenguaje preentrenado en un conjunto de datos específico para mejorar su comportamiento en contextos particulares. Este proceso optimiza la efectividad del modelo en aplicaciones industriales específicas, como en medicina, derecho o ingeniería. Sin embargo, una desventaja del *fine-tuning* es que el modelo resultante puede quedar congelado en un estado particular, lo que limita su adaptabilidad a nuevos contextos sin un nuevo ciclo de entrenamiento. Aunque el *fine-tuning* proporciona respuestas más precisas en contextos específicos, puede carecer de la flexibilidad de RAG, que permite incorporar datos contextuales en tiempo real.

2.3.3. Ventajas y desventajas de los modelos RAG y Fine-Tuning

A continuación, se presenta en la tabla 2.1 las comparaciones de las principales ventajas y desventajas de los modelos RAG (por sus siglas en inglés, *Retrieval-Augmented Generation*) y *Fine-Tuning*. Esta comparación proporciona una visión clara de las características distintivas de cada enfoque, lo que permite una evaluación informada de su idoneidad en función de los requisitos específicos del entorno de implementación.

TABLA 2.1. Ventajas y desventajas de los modelos RAG y *Fine-Tuning*.

Modelo	Ventajas	Desventajas
RAG	■ Permite incorporar información contextual actualizada en tiempo real. ■ Reduce la probabilidad de alucinaciones al usar datos relevantes. ■ Mejora la precisión y relevancia de las respuestas del chatbot.	■ Requiere una infraestructura adecuada para la recuperación de información. ■ Dependencia de la calidad y disponibilidad de la base de datos. ■ Puede ser más complejo de implementar y mantener.
<i>Fine-Tuning</i>	■ Mejora la calidad de las respuestas al adaptar el modelo a un dominio específico. ■ Proporciona un rendimiento más consistente en contextos bien definidos. ■ Permite la optimización de la latencia al trabajar con un modelo entrenado.	■ El modelo se congela en el tiempo y no se adapta a cambios contextuales. ■ Requiere un conjunto de datos específico para el entrenamiento, lo que puede ser costoso. ■ Menor flexibilidad para abordar consultas inesperadas o no entrenadas.

Capítulo 3

Diseño e implementación

3.1. Diseño de la arquitectura

En este capítulo se expone el proceso de diseño e implementación del sistema chatbot propuesto. Se analiza la arquitectura general, desde las alternativas evaluadas hasta la solución seleccionada, junto con los problemas técnicos identificados y las soluciones implementadas. Asimismo, se describen las tecnologías empleadas y el funcionamiento general del sistema. Posteriormente, se presentan las etapas clave para su implementación, la evaluación automatizada y el proceso de despliegue del sistema.

3.1.1. Propuestas de implementación

Durante el desarrollo del sistema se elaboraron cuatro versiones principales, cada una representa un cambio significativo en la tecnología, la arquitectura o el enfoque funcional del chatbot. A continuación, se describen dichas iteraciones:

- Versión 0 - RAG básico con Pinecone.

La necesidad inicial consistía en construir un prototipo funcional para validar la arquitectura RAG mediante herramientas accesibles.

Descripción: se implementó una arquitectura RAG elemental que utilizó un modelo de lenguaje grande (LLM), específicamente GPT-3.5-turbo, junto con el servicio de almacenamiento vectorial Pinecone [5]. El flujo básico consistió en preprocesar un documento extenso que contenía toda la información relevante del dominio. Se aplicó una lógica sencilla de particionado en secciones temáticas o por longitud, con el fin de facilitar su procesamiento. Cada sección fue transformada en un *embedding* y almacenada en Pinecone. Posteriormente, ante una consulta del usuario, se generaba el *embedding* correspondiente a la pregunta y se realizaba una búsqueda semántica en el *vectorstore* para recuperar los fragmentos más relevantes. Estos fragmentos se concatenaron al *prompt*, lo que enriqueció el contexto enviado al modelo de lenguaje para generar respuestas más precisas y alineadas con el contenido original.

Problemas detectados: aunque Pinecone ofrecía un buen desempeño en la recuperación de información, su costo por consulta representaba un obstáculo considerable para su adopción en entornos de producción. Además,

la trazabilidad del sistema era limitada, ya que no existían mecanismos para evaluar la calidad de los fragmentos obtenidos ni auditar el proceso de recuperación.

Estos problemas, pusieron de manifiesto la necesidad de un sistema de almacenamiento local, que permitiera un mayor control sobre la generación, almacenamiento y recuperación de los *embeddings*. Asimismo, se requerían herramientas que facilitaran tanto la evaluación de las respuestas como la auditoría del flujo completo de recuperación de contextos relevantes.

- Versión 1 - Integración de DSPy, migración a LanceDB y uso de NeMo Guardrails.

Razón del cambio: mejorar la calidad de las respuestas, reducir los costos de infraestructura y establecer una capa de control conversacional previa a la ejecución del *pipeline* RAG.

Descripción: en esta versión se reemplazó Pinecone por LanceDB, una base de datos de vectores que permite almacenamiento local y proporciona mayor control sobre la estructura y el acceso a los *embeddings*, lo que elimina los costos asociados a servicios externos.

Se integró DSPy, un *framework* que permite definir *prompts* de manera modular y declarativa, lo que facilitó una implementación más mantenible y escalable de la arquitectura RAG.

Asimismo, se añadió una capa de filtrado con NeMo Guardrails [6] para bloquear preguntas fuera de dominio, junto con un sistema de evaluación automática que puntuaba las respuestas de la LLM, y permitía monitorear su rendimiento. Estos componentes fueron implementados como módulos independientes en DSPy, lo que mejoró la modularidad y claridad del sistema. La figura 3.1 muestra el esquema general del sistema desarrollado en esta versión, y destaca los componentes principales y su integración en el flujo conversacional.

Problemas detectados: aunque se logró mayor robustez y control, surgieron desafíos como el rendimiento limitado de NeMo Guardrails en español y la falta de trazabilidad completa, lo que motivó la búsqueda de alternativas.

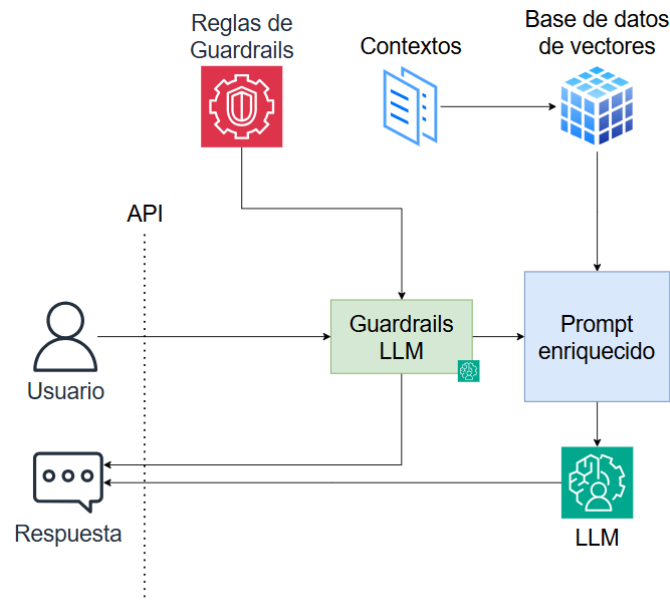


FIGURA 3.1. Diagrama general del sistema para la versión 1.

- Versión 2 - *Guardrails* personalizados, almacenamiento estructurado de chats, implementación de sistema de métricas e implementación del frontend y backend de pruebas.

Razón del cambio: necesidad de una mayor precisión en el filtrado de preguntas fuera de contexto, mejor trazabilidad de las conversaciones y cálculo de métricas para análisis de performance.

Descripción: se reemplazó NeMo *Guardrails* por una implementación propia, basada en LLMs livianos y reglas personalizadas escritas en código con *prompts* específicos. Esto permitió una mayor flexibilidad y una mejor adaptación al idioma español.

Se comenzó a almacenar localmente la conversación completa de cada usuario, con preguntas, respuestas y el contexto previo, en la memoria RAM. Esta funcionalidad dotó al sistema de memoria conversacional, lo que mejoró la coherencia de las respuestas. En paralelo, todas las interacciones del asistente (preguntas, respuestas, errores y rechazos por *guardrails*) comenzaron a registrarse en una base de datos NoSQL. Esta práctica permitió una mayor trazabilidad de cada interacción y facilitó el análisis detallado del comportamiento conversacional, como se muestra en la figura 3.2. Gracias a esto, se pudo implementar un sistema de métricas que permitía calcular estadísticas de rendimiento y analizar patrones de uso.

Descripción del sistema de métricas: la solución implementada permitió analizar automáticamente los mensajes recibidos por el asistente y clasificarlos según su intención, mediante una combinación de tecnologías de procesamiento de lenguaje natural (PLN). La arquitectura se basó en un sistema de clasificación híbrido que empleó modelos de lenguaje como DistilBERT multilingüe (a través de la biblioteca Hugging Face), FastText para el análisis vectorial de lenguaje coloquial y Sentence Transformers para la detección de similitud semántica. Cada mensaje fue evaluado por estos

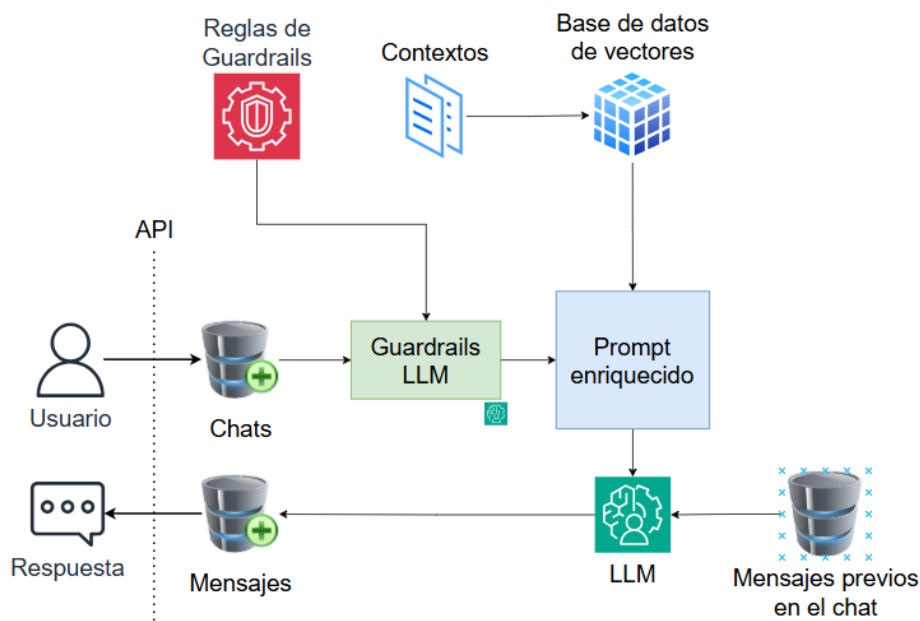


FIGURA 3.2. Diagrama general del sistema para la versión 2.

tres modelos, y la decisión final se determinó mediante un mecanismo de votación basado en el mayor puntaje. Esta combinación mejoró la precisión del sistema frente a distintos tipos de expresiones, registros lingüísticos y errores comunes en el texto.

Una vez clasificados, los mensajes fueron almacenados de forma estructurada y pudieron someterse a procesos de `clustering`. Para ello, se emplearon técnicas de TF-IDF y el algoritmo K-Means, con la eliminación de palabras vacías (`stopwords`) mediante la biblioteca NLTK. Esto permitió segmentar, por ejemplo, las consultas sobre productos en temas más específicos, lo que facilitó el análisis posterior y la detección de tendencias.

El resultado fue un sistema que no solo clasificó los mensajes con alta precisión, sino que también permitió la exploración temática del contenido agrupado. En la figura 3.3 se presenta un ejemplo gráfico de clasificación y agrupación:

Problemas detectados: aunque el nuevo sistema de *guardrails* ofreció un mayor control, su mecanismo de almacenamiento de historiales incrementó el uso de RAM y la latencia. Esta situación afectó tanto la escalabilidad como la experiencia del usuario. Además, en las preguntas encadenadas, el sistema no siempre interpretaba correctamente el contexto.

Por ejemplo:

- Usuario: "¿Tienen stock del producto X?"
- Robot: "Sí, tenemos stock del producto X."
- Usuario: "¿Cuánto cuesta?"
- Robot: "¿Podrías aclarar de qué producto hablas?"

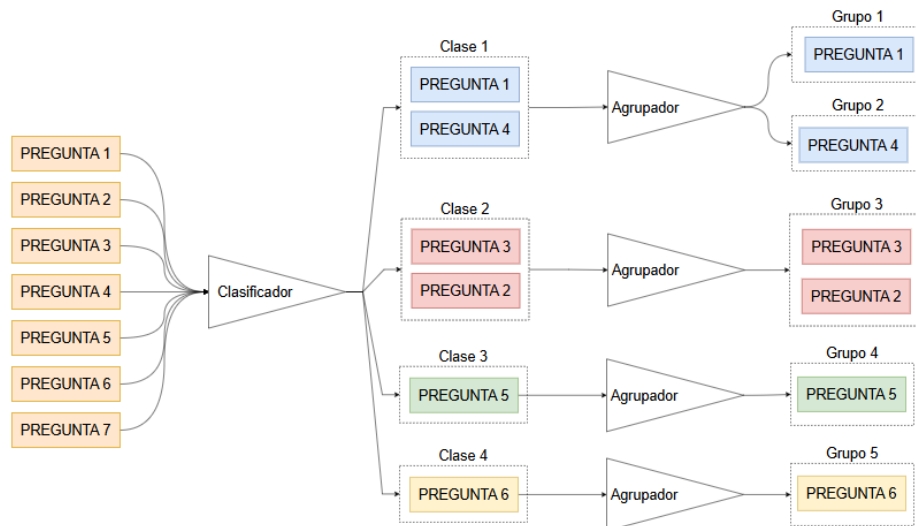


FIGURA 3.3. Diagrama general del sistema de métricas de la versión 2.

En este caso, el sistema no logró identificar que la pregunta hacía referencia al producto mencionado previamente. Aunque se empleaba el historial de conversación para proporcionar contexto a la LLM, el mecanismo de recuperación de información dependía exclusivamente de la formulación de la pregunta actual. Esta limitación evidenció la necesidad de incorporar un sistema de reformulación de preguntas que enriqueciera el contexto conversacional. Este proceso no debía limitarse al uso de palabras clave, sino que también debía integrar referencias explícitas a intervenciones anteriores. Por ejemplo, en el caso previamente analizado, la búsqueda de contextos se iniciaba a partir de la consulta “¿cuánto cuesta?”. Sin embargo, lo adecuado habría sido reformular la pregunta a “¿cuánto cuesta el producto X?”, ya que esa versión incluye la referencia al elemento mencionado anteriormente. De este modo, las búsquedas semánticas podrían orientarse hacia contextos más coherentes y relevantes, alineados con la continuidad temática de la interacción.

Por otro lado, el sistema de métricas, aunque útil para el análisis de performance, presentó limitaciones: el uso simultáneo de múltiples modelos incrementó el tiempo de respuesta y el consumo de cómputo, y la clasificación dependía demasiado de ejemplos predefinidos, lo que reducía su robustez frente a consultas poco comunes o mal redactadas.

- Versión 3 - Integración de Langfuse, clasificación y evaluación automática.

Razón del cambio: necesidad de un sistema robusto para trazabilidad, evaluación y mejora continua.

Descripción: se integró Langfuse como sistema observador no intrusivo para registrar todas las interacciones del sistema. Asimismo, se incorporaron dos nuevas tareas en la capa de restricciones (*guardrails*). La primera consistió en la reformulación de preguntas, que generaba una nueva oración con palabras clave relacionadas con el contexto de la consulta actual, con el fin de relacionar preguntas anidadas, como se muestra en el ejemplo de la figura 3.4. Este proceso tenía como objetivo optimizar la búsqueda de contextos



FIGURA 3.4. Ejemplo de redefinición de pregunta.

en el *vectorstore*. La segunda tarea correspondió a la clasificación de consultas, encargada de categorizarlas según su temática, tales como productos, precios, delivery, quejas, saludos, entre otras. Además, se eliminaron las bases de datos auxiliares y se centralizó la gestión de trazabilidad en Langfuse, que pasó a encargarse del registro de todos los eventos relevantes del sistema: consultas, respuestas, errores, rechazos por restricciones, consumo de tokens, costos, latencia, entre otros.

Problemas detectados: algunos temas, como las quejas, mostraron un rendimiento inferior, lo que sugiere la necesidad de expandir y refinar el contexto disponible en versiones futuras.

3.1.2. Problemas y mitigaciones

Durante las primeras iteraciones surgieron limitaciones clave, como costos por servicios externos (Pinecone, API de LLMs), pobre trazabilidad del sistema y ausencia de control sobre la evaluación de calidad. Estos problemas fueron mitigados mediante las siguientes acciones:

- Migración a LanceDB para almacenamiento local sin costo: LanceDB es una base de datos vectorial optimizada para uso local. Su adopción permitió eliminar la dependencia de servicios como Pinecone, que implicaban costos por cada consulta. Esta decisión fue especialmente útil durante la etapa de desarrollo y pruebas, ya que redujo los costos operativos. Sin embargo, se observó que LanceDB puede presentar latencias más altas en comparación con soluciones SaaS (por sus siglas en inglés, *Software as a Service*) [7], que deben considerarse en entornos de producción. Por otro lado, para aplicaciones a gran escala, su uso local podría representar un ahorro sustancial al evitar tarifas por tokens o el uso de APIs externas.
- Uso de DSPy para estructurar la lógica RAG y mejorar la modularidad: DSPy es una biblioteca que permite construir y optimizar *pipelines* de IA mediante programación declarativa. Se utilizó para implementar de manera modular el sistema RAG, y se definieron claramente las etapas de recuperación y generación. Esta implementación facilitó la trazabilidad del proceso completo, y permitió auditar tanto la recuperación de contextos como la generación de respuestas. Además, su enfoque declarativo simplificó el desarrollo, mejoró la mantenibilidad y facilitó la introducción de mejoras incrementales al sistema.
- Reemplazo de NeMo Guardrails por *guardrails* personalizados: se diseñó una solución propia basada en *prompts* simples con modelos LLM. Esta implementación personalizada fue capaz de detectar y filtrar preguntas sobre política, religión y otros temas sensibles. Además, se utilizó como etapa inicial para la clasificación temática y la redefinición de preguntas en etapas posteriores.
- Incorporación de Langfuse como observador centralizado y sistema de métricas: Langfuse es una herramienta de observabilidad que permite registrar, analizar y etiquetar cada interacción con el modelo. Su integración aportó trazabilidad completa al sistema, facilitó el monitoreo de métricas clave (como latencia, consumo de tokens, respuestas generadas) y habilitó la evaluación automática mediante la implementación de agentes de evaluación con IA en su plataforma. Gracias a esta integración, se logró una visión integral del rendimiento del sistema, lo que permitió identificar áreas de mejora y optimizar el flujo de trabajo. Esta capacidad eliminó la etapa de DSPy que se encargaba de la evaluación automática posterior a la respuesta de la LLM. Esto redujo la latencia del sistema y permitió una mejor trazabilidad de las interacciones, ya que Langfuse almacena todos los eventos generados por el sistema, con la evaluación automática. Esto facilita la auditoría del funcionamiento del sistema y mejora el contexto disponible en la etapa de análisis de métricas.

3.1.3. Elección de la arquitectura de chatbot

La arquitectura final del chatbot se basa en el enfoque RAG (por sus siglas en inglés, *Retrieval-Augmented Generation*), una técnica que ha demostrado ser altamente efectiva para responder preguntas contextualizadas sin necesidad de realizar un reentrenamiento completo del modelo de lenguaje (LLM). Esta decisión se tomó para ofrecer un sistema flexible, escalable y de bajo costo operativo, especialmente orientado a pequeñas empresas y emprendedores que requieren soluciones ágiles y sostenibles.

El uso de RAG permite mantener el modelo LLM independiente de la base de conocimiento. Esto implica que se pueden incorporar nuevos documentos, actualizaciones o correcciones mediante la modificación de los datos almacenados en el *vectorstore*, sin necesidad de ajustar los parámetros del modelo. Esta separación entre el modelo y la información facilita considerablemente las tareas de mantenimiento y mejora continua del sistema.

La elección de esta arquitectura RAG se fundamentó en su capacidad para ofrecer un equilibrio óptimo entre precisión, costo y mantenibilidad. Durante el desarrollo del proyecto, OpenAI lanzó GPT-4o mini, un modelo que presentó un excelente balance entre rendimiento y economía, con capacidades superiores a las alternativas locales disponibles en ese momento. Aunque se evaluó la posibilidad de implementar un modelo local con Ollama [8], la combinación de GPT-4o mini con LanceDB resultó ser más eficiente en términos de costo-beneficio, ya que eliminaba la necesidad de infraestructura adicional para hospedar y mantener modelos locales. Esta decisión redujo significativamente el costo por consulta, y mantuvo un alto estándar de calidad en las respuestas, lo que hizo que la solución fuera más accesible para pequeñas empresas y emprendedores. Además, el uso de componentes modulares como DSPy facilitó la adaptación del sistema a nuevos modelos o proveedores de LLMs en el futuro, sin necesidad de rediseñar toda la arquitectura.

3.2. Arquitectura final propuesta

La arquitectura final del sistema está diseñada para ofrecer una solución robusta, eficiente y fácilmente escalable para la automatización de respuestas contextuales. Está compuesta por los siguientes bloques funcionales:

- **Frontend:** interfaz renovada para usuarios y administradores. Permite enviar preguntas al sistema, visualizar respuestas, acceder a métricas y gestionar los contextos mediante un formulario ilustrativo.
- **Backend:** implementado en FastAPI [9], se encarga de recibir las solicitudes del frontend para procesar las preguntas, enviar las respuestas, gestionar las métricas requeridas y actualizar los contextos en la base de datos de vectores.
- **Guardrails:** módulo de filtrado y preprocesamiento que evalúa si una consulta es válida (según reglas predefinidas), la clasifica en categorías como “Producto”, “Quejas” o “Saludos” y la redefine con el contexto de los mensajes anteriores.
- **DSPy RAG:** responsable de orquestar la lógica principal del flujo RAG. DSPy toma la pregunta ya enriquecida desde el módulo de *guardrails* y ejecuta

una búsqueda eficiente de contextos relevantes en LanceDB. A partir de los fragmentos recuperados, construye un *prompt* enriquecido que será enviado al modelo LLM para generar una respuesta coherente y precisa. Además de facilitar la integración de *prompts*, DSPY permite definir y estructurar toda la lógica del sistema de recuperación y generación, y proporciona un marco flexible y modular para el diseño del flujo completo.

- Langfuse: observador pasivo que registra todas las interacciones del sistema sin intervenir. Almacena información crítica como la pregunta original, la redefinida, el consumo de tokens, la respuesta final y los estados de los *guardrails*. También permite aplicar evaluaciones automáticas sobre la precisión de las respuestas generadas por el sistema que luego son usadas por el sistema de métricas.

La figura 3.5 muestra el diagrama de la arquitectura completa, en la que se detallan los principales componentes y flujos de información del sistema.

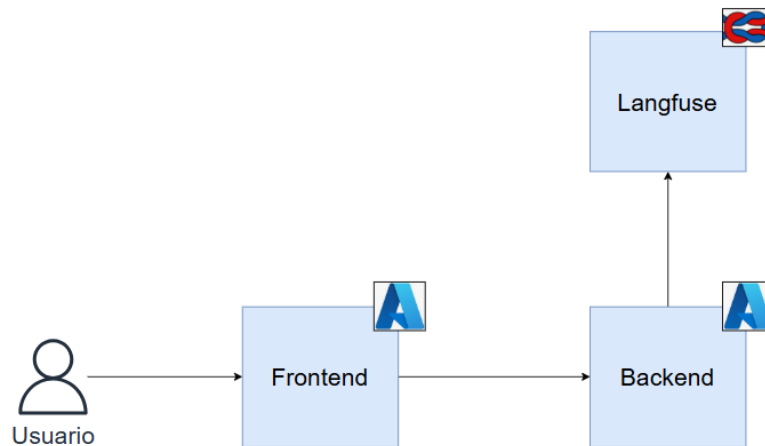


FIGURA 3.5. Diagrama general del flujo del *pipeline* del sistema.

3.2.1. Tecnologías suplementarias

- *Streamlit*: utilizado para construir el frontend inicial de pruebas, y permite una interacción simple con el chat, que posteriormente fue migrada a ReactJS. *Streamlit* es un *framework* de código abierto en Python para crear aplicaciones web de forma rápida y sencilla, especialmente orientado a proyectos simples o pruebas de concepto.
- *lovable.dev*: es una herramienta en línea que permite crear frontends de forma rápida y sencilla con la ayuda de inteligencia artificial. Fue utilizada para desarrollar la versión final del frontend con ReactJS, y proporcionó una experiencia de usuario más fluida y atractiva. Además, en esta plataforma se integraron las visualizaciones de las métricas generadas a partir de los datos de Langfuse.
- *Azure Container Apps*: recurso de la plataforma Azure diseñado para ejecutar aplicaciones en contenedores sin necesidad de administrar directamente la infraestructura. Fue utilizado para desplegar tanto el backend como el frontend del sistema en contenedores individuales, lo que permite escalar automáticamente según la demanda y asegura un rendimiento óptimo.

3.2.2. Funcionamiento general

El *pipeline* del sistema está diseñado para ofrecer una experiencia conversacional precisa, eficiente y trazable. A continuación, se detalla el flujo completo de funcionamiento representado en la figura 3.6:

1. El usuario realiza una consulta a través del frontend.
2. La consulta es enviada al backend, donde primero pasa por un sistema de *guardrails* personalizados. En este proceso se ejecutan tres tareas clave:
 - Filtrado: se descartan preguntas fuera del dominio del sistema (por ejemplo, política o religión), basándose en reglas predefinidas.

- Clasificación: la pregunta se categoriza automáticamente según su temática (Producto, Quejas, Saludos, etc.).
 - Redefinición: con el historial conversacional completo, la pregunta se reformula para enriquecer su significado y contexto.
3. La pregunta redefinida es ingresada al primer módulo de `DSPy`, donde se utiliza para buscar en `LanceDB` los fragmentos de contexto indexados previamente que están relacionados con la pregunta en cuestión.
 4. La información obtenida se utiliza para construir un *prompt* estructurado en el siguiente módulo de `DSPy`. Esta etapa representa el componente fundamental del sistema RAG dentro del *pipeline*. En esta fase, tanto el contexto recuperado como la consulta original se integran para preparar la entrada final al modelo LLM.
 5. El *prompt* es enviado a un modelo LLM (en este caso, GPT-4o mini).
 6. La respuesta y todos los datos asociados (estado del guardrail, redefinición, tokens consumidos, clasificación, etc.) son registrados por Langfuse.
 7. En Langfuse, se ejecuta una evaluación automática, que analiza la calidad de la respuesta y asigna puntajes en una escala del 1 al 10. Los criterios de evaluación son:
 - Precisión: determina si la respuesta aborda directamente la pregunta del usuario o simplemente la reformula sin proporcionar información sustancial.
 - Suficiencia de contexto: evalúa si la respuesta demuestra que el sistema disponía de información adecuada para contestar con propiedad o si indica explícitamente la falta de contexto necesario.
 8. En paralelo con la observación de Langfuse, la respuesta es devuelta al usuario en el frontend.

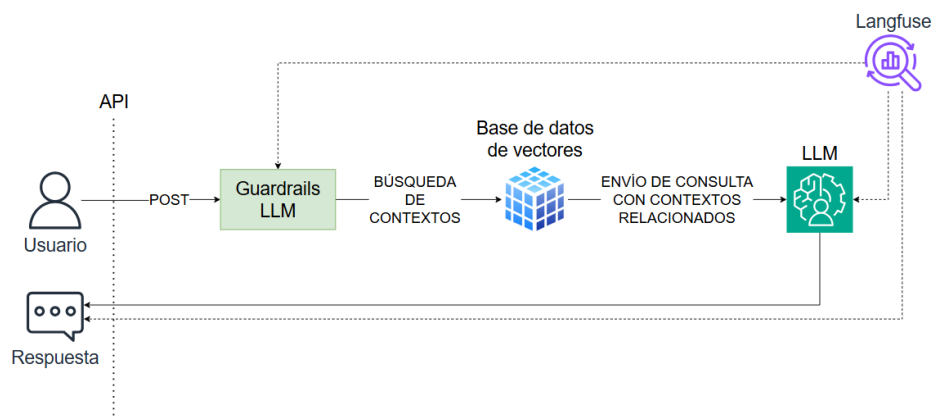


FIGURA 3.6. Diagrama de flujo de funcionamiento.

3.3. Implementación

3.3.1. Preparación de los datos de contexto

La base de conocimiento utilizada por el sistema no se deriva de documentos extensos divididos en segmentos, como en versiones anteriores. En su lugar, se construyó a partir de una estructura de tipo clave-valor, diseñada específicamente para permitir búsquedas contextuales rápidas y precisas. Cada entrada en esta estructura contiene dos campos principales:

- **question**: una lista de términos, sinónimos o expresiones relacionadas que representan la intención del usuario. Este campo actúa como una clave semántica para la búsqueda.
- **relates**: una serie de frases informativas asociadas a la clave, que constituyen el contexto que será devuelto al modelo como parte del proceso RAG.

La estructura clave-valor facilita la búsqueda semántica eficiente, y permite que el sistema recupere contextos relevantes rápidamente al comparar las consultas del usuario con las claves almacenadas en la base de datos. La figura 3.7 muestra un ejemplo de cómo se estructuran estos datos.

```
CONTEXT_DATA = [
{
  "question": "Hola, hello, buenas tardes, saludo, bienvenido, bienvenida",
  "relates": """Saludos:
1. Hola Bienvenido a PetArg Boutique 🐾
2. Hola, amigo de las mascotas Gracias por visitarnos en PetArg Boutique.
3. 🐾 Bienvenido ¿Buscás lo mejor para tu mascota? Estás en el lugar correcto.
4. Bienvenido a PetArg Boutique ¿En qué te podemos ayudar hoy?"""
},
.....
{
  "question": """Servicios, peluquería, chequeos, vacunaciones,
entrenamiento, alojamiento, cuidados, atención veterinaria,
entrenamiento de mascotas, guardería, hospedaje""",
  "relates": """Servicios: \n\tEn PetArg Boutique, ofrecemos una ampliagama
de servicios personalizados para cada mascota: \n\t
1. 🐾 Peluquería: \n\t\t- Corte y baño según la raza y el
estilo de tu mascota. \n\t\t- Cuidado de uñas y limpieza de oídos. \n\t\t-
Aromaterapia para reducir el estrés durante la sesión. \n\t
2. 🐾 Chequeos de salud y vacunaciones: \n\t\t- Control veterinario
general. \n\t\t- Calendario de vacunación completo. \n\t\t-
Tratamientos antiparasitarios. \n\t
3. 🐾 Entrenamiento: \n\t\t-
Obediencia básica. \n\t\t- Modificación de conductas no deseadas. \n\t\t-
Entrenamiento avanzado y socialización para cachorros. \n\t
4. 🐾 Alojamiento: \n\t\t- Guardería de día con juegos supervisados. \n\t\t-
Alojamiento nocturno seguro y cómodo. \n\t\t-
Atención personalizada las 24 horas.""",
}
```

FIGURA 3.7. Ejemplo de formato de contexto.

Esta estructura permite una consulta eficiente, ya que las claves semánticas generadas durante el proceso de reformulación se emplean para buscar y recuperar los fragmentos de contexto más adecuados. A partir de estas claves, se extraen las frases informativas ubicadas en el campo **relates**. La búsqueda se realiza mediante un enfoque híbrido que combina técnicas semánticas y vectoriales, lo que permite identificar coincidencias exactas y también similitudes conceptuales entre la consulta y las claves almacenadas. Una vez identificadas las coincidencias, LanceDB accede a los valores del campo **relates**, que contienen el contexto necesario para interpretar correctamente la consulta. Después, el sistema inserta los

fragmentos de contexto relevantes en el *prompt* del modelo generador LLM. Este procedimiento garantiza respuestas precisas y contextualizadas, al integrar tanto la consulta original del usuario como el contexto

3.3.2. Métricas, clasificadores y análisis semántico en sistemas conversacionales

La evaluación precisa y continua de los sistemas conversacionales es esencial para garantizar respuestas de alta calidad y mantener la relevancia en interacciones dinámicas con usuarios. En el contexto de la inteligencia artificial aplicada a chatbots y asistentes virtuales, las métricas cuantitativas y cualitativas juegan un rol fundamental para medir el desempeño, identificar áreas de mejora y orientar estrategias de optimización.

Las métricas pueden abarcar desde indicadores técnicos, como el consumo de recursos y la latencia, hasta evaluaciones de calidad semántica, coherencia y satisfacción del usuario. Para complementar estas mediciones, se utilizan clasificadores automáticos que categorizan las interacciones en función de etiquetas temáticas o niveles de calidad. Esto facilita la segmentación y el análisis detallado de los datos conversacionales.

Sin embargo, el análisis superficial basado en etiquetas o puntuaciones individuales resulta insuficiente para comprender la complejidad y diversidad de las preguntas que recibe el sistema. Es aquí donde las técnicas avanzadas de procesamiento de lenguaje natural (NLP) y el aprendizaje no supervisado cobran especial relevancia y permiten transformar textos en representaciones numéricas llamadas *embeddings*, que capturan las relaciones semánticas entre las diferentes expresiones lingüísticas.

Estos *embeddings* habilitan la aplicación de métodos de agrupamiento semántico, que buscan identificar patrones y similitudes en grandes volúmenes de datos no estructurados. El análisis de estos grupos permite descubrir temas recurrentes, detectar desviaciones o inconsistencias, y orientar ajustes precisos en la configuración del sistema.

En particular, el uso de algoritmos basados en densidad, como HDBSCAN, ofrece ventajas significativas para agrupar datos de naturaleza compleja y heterogénea. Estos algoritmos no requieren predefinir la cantidad de grupos y manejan eficazmente el ruido o las preguntas atípicas. Esta característica facilita una visión más profunda y detallada del comportamiento del sistema, lo que resulta clave para implementar procesos de mejora continua. Estos procesos no dependen exclusivamente del reentrenamiento de modelos de lenguaje, sino que aprovechan la riqueza de los datos conversacionales reales.

3.3.3. Sistema de evaluación automatizada y mejora continua sin reentrenamiento

Durante el desarrollo del sistema, se adoptó un enfoque alternativo al reentrenamiento tradicional de modelos de lenguaje (LLM). Se priorizó la adaptabilidad mediante evaluación automatizada y actualización contextual. Esta arquitectura permite analizar los resultados de las interacciones con los usuarios sin modificar directamente los parámetros del modelo base.

En lugar de incorporar nuevos datos etiquetados para ajustar el modelo, se estableció un ciclo de mejora continua basado en datos conversacionales reales. Dichos datos se recolectan, organizan y evalúan de forma semiautomatizada, a través de herramientas diseñadas para monitorear el rendimiento y la calidad de las interacciones. Este proceso resulta más eficiente y flexible para entornos en producción, ya que evita intervenciones complejas y costosas.

Cada interacción registrada genera una traza almacenada con Langfuse, que incluye los mensajes de entrada y salida, consumo de tokens, costo asociado, etiquetas temáticas obtenidas durante la etapa de restricciones y puntajes de calidad en una escala del 1 al 10. Esta trazabilidad proporciona visibilidad crítica para la toma de decisiones orientadas a la optimización del sistema.

El administrador revisa las métricas generadas desde una interfaz en el frontend, donde puede gestionar la información contextual sin alterar el modelo base. Este mecanismo permite detectar patrones relevantes, evaluar la coherencia semántica de las respuestas y ajustar el comportamiento general del sistema. Durante el proceso de evaluación de las métricas, se recorrieron todas las trazas válidas y se excluyeron aquellas clasificadas como "saludo", además se identificaron las que contienen puntuaciones explícitas. Estas trazas se procesan mediante un *pipeline* estructurado compuesto por las siguientes etapas:

1. Preprocesamiento del texto: las preguntas se normalizan mediante la conversión a minúsculas, la eliminación de acentos y de símbolos no relevantes, con el fin de garantizar consistencia en el análisis posterior.
2. Asociación temática: las preguntas se agrupan según las etiquetas temáticas estandarizadas incluidas en su metainformación.
3. Evaluación cuantitativa: se analizan los puntajes de calidad asignados por el agente de Langfuse. A partir de estos valores, se calculan promedios y se clasifican las preguntas en dos categorías: buenas (≥ 8) y malas (< 5).
4. Agrupamiento semántico: las preguntas se transforman en vectores de *embeddings* mediante técnicas de procesamiento de lenguaje natural (PLN). Posteriormente, se agrupan con HDBSCAN [10], con el objetivo de identificar similitudes semánticas y patrones recurrentes.
5. Síntesis por grupo: se selecciona una pregunta representativa de cada *cluster* mayoritario, a partir de la cual se genera un resumen conciso para cada etiqueta temática. Esta síntesis facilita la visualización del desempeño del sistema y la identificación de áreas susceptibles de mejora.

En la figura 3.8 se presenta un diagrama del *pipeline* que ilustra el proceso de extracción de las métricas desde Langfuse, el agrupamiento en el backend y la posterior renderización en el frontend.

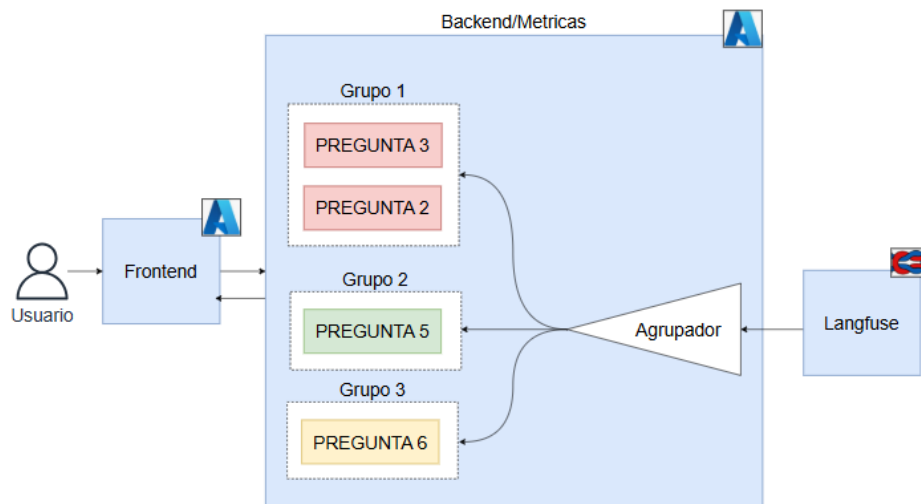


FIGURA 3.8. Diagrama de flujo de observación de métricas.

3.3.4. Despliegue del sistema

El sistema se desplegó en *Azure Container Apps*, una plataforma que permite ejecutar contenedores en la nube con escalabilidad automática y administración simplificada. Esta solución ofrece una gestión eficiente de los recursos y garantiza un rendimiento constante, incluso ante variaciones en la carga de trabajo.

Una de sus principales ventajas es que no requiere gestionar directamente la infraestructura subyacente, lo que permite al equipo de desarrollo enfocarse en la evolución funcional del sistema en lugar de tareas operativas. Asimismo, la plataforma facilita las tareas de mantenimiento y actualización, lo que acelera los ciclos de mejora continua.

Azure Container Apps también ofrece una integración nativa con otros servicios del ecosistema Azure, lo que habilita la incorporación fluida de nuevas funcionalidades y simplifica la expansión del sistema en escenarios futuros.

Capítulo 4

Ensayos y resultados

En este capítulo se presentan las pruebas realizadas al chatbot desarrollado, junto con los resultados obtenidos en distintos escenarios. Se detallan los procesos de optimización aplicados, se analiza el comportamiento del sistema y se identifican oportunidades de mejora. Asimismo, se expone la metodología de evaluación empleada y se discuten los hallazgos más relevantes, con el objetivo de orientar futuras iteraciones del sistema.

4.1. Evaluación de calidad en sistemas conversacionales

La evaluación de la calidad en sistemas conversacionales basados en modelos de lenguaje es fundamental para garantizar que las respuestas generadas sean útiles, coherentes, contextualizadas y acordes con la intención del usuario. A diferencia de los sistemas deterministas clásicos, los modelos generativos producen salidas de naturaleza probabilística. Esto exige métodos de evaluación específicos, tanto automáticos como humanos, que permitan analizar la experiencia del usuario y la efectividad del sistema.

4.1.1. Criterios y mecanismos de evaluación

La determinación de la calidad de una respuesta presenta un alto grado de complejidad. Para su valoración, se han definido criterios ampliamente aceptados que constituyen la base de la evaluación:

- Relevancia: la respuesta debe estar directamente relacionada con la consulta del usuario.
- Claridad y coherencia: debe expresarse de forma comprensible y sin contradicciones.
- Contextualización: debe considerar el historial conversacional y adaptarse adecuadamente a la situación planteada.

Dado que estos aspectos no siempre pueden medirse mediante métricas automáticas, se adopta un enfoque híbrido que combina herramientas automatizadas con revisión humana, con el fin de lograr una evaluación más integral.

4.1.2. Evaluación, observabilidad y trazabilidad

El sistema adopta un enfoque híbrido para evaluar la calidad de las respuestas. Combina mecanismos automatizados y revisión humana, complementados con capacidades de observabilidad y trazabilidad para la mejora continua.

Langfuse se utiliza como plataforma central para el registro y análisis de interacciones. Integra agentes automáticos que asignan puntuaciones de calidad en una escala del 1 al 10, basadas en reglas y modelos predefinidos. Estas puntuaciones, almacenadas junto con las trazas conversacionales, permiten detectar respuestas problemáticas, identificar patrones de error mediante agrupamiento semántico y calcular métricas según el tipo de pregunta, contexto o modelo. Esto facilita la escalabilidad del control de calidad sin supervisión constante.

Como complemento cualitativo, el administrador realiza revisiones manuales enfocadas en casos ambiguos que no se resuelven automáticamente, preguntas frecuentes con baja puntuación que requieren ajustes, y oportunidades de mejora en el tono o la empatía conversacional.

4.2. Despliegue de la interfaz de pruebas

Se desarrolló una interfaz interactiva con `ReactJS`, con apoyo en las herramientas en línea de `lovable.dev`. La interfaz tuvo un doble propósito: facilitar las pruebas de usuario y permitir la administración dinámica de los contextos del sistema.

4.2.1. Interfaz de usuario

Chatbot

La interfaz de usuario permite realizar preguntas al chatbot y visualizar las respuestas en tiempo real, lo que habilita una interacción directa con el sistema y un análisis inmediato de la calidad de las respuestas. Presenta un diseño intuitivo y accesible, compuesto por un campo de texto para ingresar preguntas, un botón de envío y un área de visualización para las respuestas generadas por el modelo. También incluye un campo de nombre, útil para la trazabilidad de usuarios durante las pruebas, ya que permitió distinguir fácilmente entre los distintos perfiles de uso, entre ellos el usuario de desarrollo empleado para pruebas locales.

En la figura 4.1 se muestra una captura de pantalla de la interfaz del chatbot, en la que se observan los principales elementos de interacción: el campo de entrada de preguntas, el botón de envío y el área de visualización de respuestas generadas por el modelo.

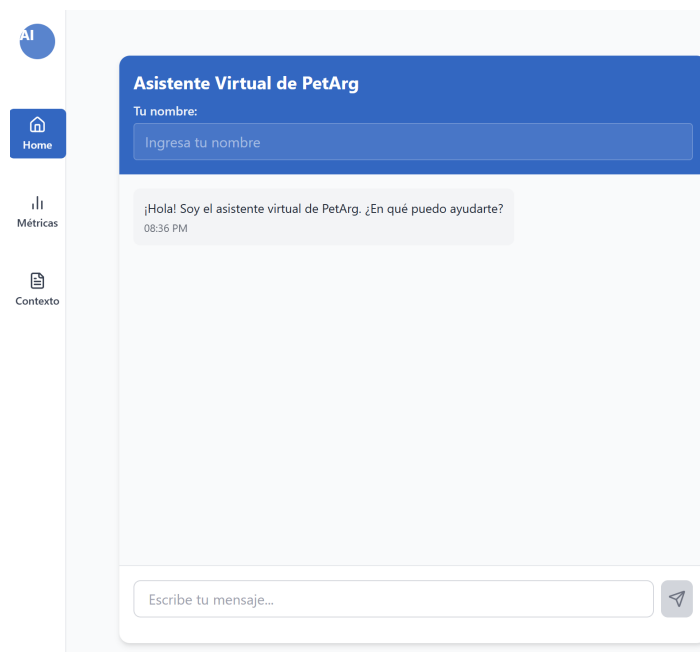


FIGURA 4.1. Frontend del chatbot.

Métricas

La interfaz permite consultar las métricas generadas por el sistema de evaluación automática, junto con las categorías temáticas asignadas a cada pregunta. Esto facilita un análisis detallado del desempeño del sistema de clasificación.

- Métricas generales: se presentan indicadores agregados del sistema, como el total de preguntas, el puntaje promedio, la desviación estándar y una evaluación global de la calidad de las respuestas generadas. Estos datos permiten identificar el rendimiento general por categoría temática y detectar áreas que requieren ajustes en el contexto. La figura 4.2 muestra la visualización de estas métricas, segmentadas por categorías, lo que facilita su interpretación y el seguimiento del comportamiento del sistema.



FIGURA 4.2. Métricas por categoría temática.

- **Buenas preguntas:** se presentan métricas específicas asociadas a preguntas que obtuvieron calificaciones altas, tales como el puntaje de calidad y un breve análisis de los factores que contribuyeron a una respuesta satisfactoria. Estas métricas son clave para identificar patrones positivos en el rendimiento del sistema, como se ilustra en la figura 4.3.

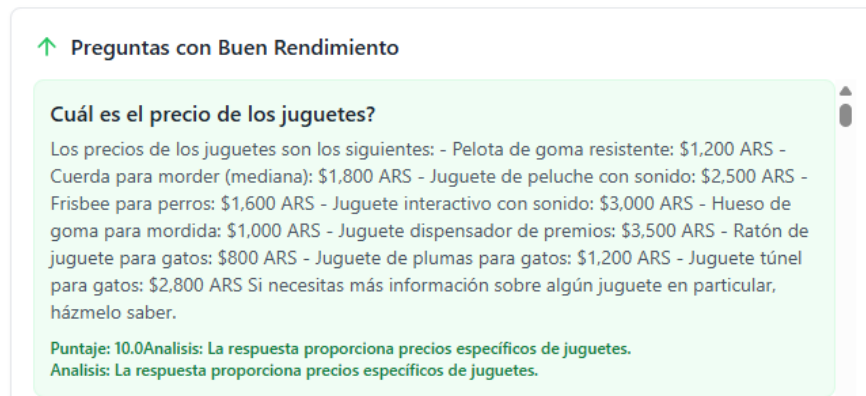


FIGURA 4.3. Ejemplo del frontend con métricas de preguntas bien calificadas.

- **Malas preguntas:** se visualizan métricas correspondientes a preguntas mal calificadas, lo que permite detectar fallos en el sistema y orientar acciones correctivas en los contextos o el flujo conversacional, como se ejemplifica en la figura 4.4.



FIGURA 4.4. Ejemplo del frontend con métricas de preguntas mal calificadas.

Administración de contextos

Esta sección de la interfaz permite modificar o agregar fragmentos de contexto mediante un formulario de carga, lo que brinda flexibilidad para actualizar la base de conocimiento de forma dinámica, sin intervención técnica directa. La figura 4.5, ilustra la vista del frontend correspondiente al formulario y las opciones disponibles para gestionar los contextos.

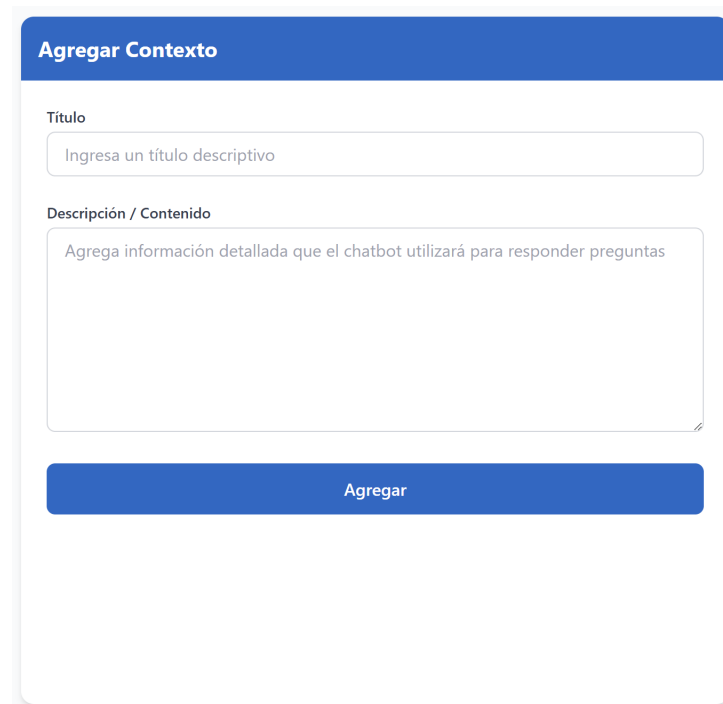
The image shows a web form titled "Agregar Contexto" in a blue header. Below the header, there is a "Titulo" section with a text input field containing the placeholder "Ingresa un título descriptivo". Underneath is a "Descripción / Contenido" section with a larger text area containing the placeholder "Agrega información detallada que el chatbot utilizará para responder preguntas". At the bottom of the form is a blue button labeled "Agregar".

FIGURA 4.5. Vista del frontend para la administración de contextos.

4.3. Evaluación de escenarios del chatbot

4.3.1. Escenario 1: interacción básica

Se evaluó la capacidad del chatbot para mantener una conversación coherente en interacciones simples. Para ello, se formularon preguntas directamente relacionadas con el contexto actual de la versión, aunque no siempre completas ni contextualizadas en el hilo conversacional. El objetivo principal de esta prueba fue verificar el denominado *happy path* del sistema; es decir, el funcionamiento correcto de las etapas del *pipeline*, que abarcan la recepción de entrada, la captura de contextos relevantes y el análisis por parte del modelo de lenguaje para generar una respuesta adecuada.

4.3.2. Escenario 2: consultas con múltiples pasos

En esta evaluación se realizaron consultas más complejas desde el punto de vista del hilo conversacional, donde el usuario planteó preguntas encadenadas que requerían del sistema la retención y gestión del contexto. Se analizó la capacidad del chatbot para mantener coherencia y relevancia a lo largo de múltiples interacciones. Si bien las preguntas fueron formuladas de forma directa, se evaluó la habilidad del sistema para adaptarse a cambios en la temática sin perder el hilo de la conversación ni mezclar las respuestas con temas tratados previamente.

4.3.3. Escenario 3: consultas fuera de contexto

En este escenario se examinó la capacidad del chatbot para gestionar preguntas ambiguas o fuera de contexto, así como su habilidad para solicitar mayor información o derivar la consulta a un humano. Cabe destacar que esta función se

encuentra en etapa de pruebas, ya que el prototipo no cuenta con una conexión real a un sistema CRM (por sus siglas en inglés, *Customer Relationship Management*) para realizar la transferencia de la conversación. El objetivo fue evaluar la habilidad del sistema para reconocer sus propias limitaciones y ofrecer respuestas apropiadas incluso ante la falta de información contextual.

4.4. Resultados de las pruebas

En este apartado se presentan los resultados obtenidos en cada uno de los escenarios de prueba descritos anteriormente para cada una de las 4 versiones del chatbot. Además, se discuten los hallazgos más relevantes y se proponen recomendaciones para futuras mejoras.

4.4.1. Pruebas para la versión 0

Pruebas de interacción básica

El sistema funcionó adecuadamente en la mayoría de las interacciones directas, y proporcionó respuestas precisas y pertinentes según la información almacenada en Pinecone. Sin embargo, debido a la simplicidad del modelo RAG y a la recuperación subóptima de fragmentos desde los *embeddings*, se detectaron dificultades ante preguntas ambiguas o imprecisas. En estos casos, las respuestas resultaron menos alineadas con la intención del usuario o presentaron imprecisiones en detalles específicos debido a la ausencia de un contexto claro.

- Hallazgos: la recuperación de información fue efectiva cuando las preguntas coincidieron con las secciones temáticas predefinidas. En contraste, las consultas formuladas de forma vaga provocaron respuestas generales.
- Recomendaciones: optimizar el preprocesamiento de documentos para atender consultas ambiguas y mejorar la calidad de los contextos retornados por el modelo de vectores.

Pruebas de consultas con múltiples pasos

El sistema no gestionó adecuadamente preguntas encadenadas o de múltiples pasos. En la mayoría de los casos, no se mantuvo el contexto conversacional y las respuestas carecieron de relación directa con la consulta inicial. Esto se debió a que el modelo RAG utilizado no contempla mecanismos para la gestión de contexto prolongado ni para el seguimiento efectivo de interacciones previas.

- Recomendaciones: incorporar un mecanismo de contexto persistente que permita conservar el hilo de la conversación, incluso ante cambios temáticos. Reforzar la recuperación semántica para atender consultas complejas de forma más precisa.

Pruebas de consultas fuera de contexto

Ante la ausencia de datos relevantes en el contexto, el sistema generó respuestas incorrectas y sin relación con la consulta. En ocasiones, se intentó responder con información irrelevante o desconectada del tema solicitado.

Conclusiones de la versión 0

La versión 0 del chatbot presentó un rendimiento adecuado en interacciones básicas y ofreció respuestas coherentes y precisas cuando las preguntas se alinearon con los fragmentos almacenados en Pinecone. Sin embargo, se identificaron limitaciones ante consultas complejas, encadenadas o fuera de contexto. Estos resultados evidencian que, aunque el sistema básico de recuperación y generación fue funcional, requería mejoras fundamentales.

4.4.2. Pruebas para la versión 1

Pruebas de interacción básica

Durante las pruebas de interacción básica, la versión 1 presentó mejoras notables en comparación con la versión anterior. La incorporación de `LanceDB` permitió una recuperación local más eficiente de fragmentos relevantes. Por otro lado, la integración de `DSPy` contribuyó a una mayor claridad en la construcción de los *prompts*, lo que facilitó respuestas más consistentes y alineadas con el contenido. En esta etapa, el sistema respondió con alta precisión a preguntas simples y directas, y cumplió las expectativas del *happy path*.

- Hallazgos: se observó una mejora en la calidad de las respuestas debido al uso de *prompts* más estructurados. La recuperación local fue eficiente y precisa, lo que redujo los tiempos de respuesta.
- Recomendaciones: ampliar los criterios de evaluación automática para considerar mayor variabilidad lingüística. Ajustar los *prompts* con el objetivo de mejorar la claridad de las respuestas en casos límite.

Pruebas de consultas con múltiples pasos

En interacciones más complejas, que incluyeron consultas encadenadas o cambios temáticos sutiles, el sistema mostró un rendimiento comparable al de la versión anterior. La retención del contexto a lo largo de múltiples turnos se mantuvo parcialmente, debido en parte a limitaciones en el flujo del *pipeline* y a la arquitectura modular, que, si bien resultó más clara, no incorporaba mecanismos explícitos para la gestión del estado conversacional.

- Recomendaciones: incorporar un componente específico para la gestión del historial conversacional dentro del *pipeline*. Explorar la integración con modelos que dispongan de memoria o buffers de contexto persistente para mejorar la continuidad de la conversación.

Pruebas de consultas fuera de contexto

Con la incorporación de `NeMo Guardrails`, el sistema identificó y bloqueó parcialmente consultas fuera de dominio, lo que generó respuestas más controladas. Adicionalmente, mediante `DSPy` se definió una regla en el *prompt* que permitía simular la derivación de la conversación hacia un asistente alternativo cuando el contexto disponible no resultaba suficiente para formular una respuesta adecuada.

- Hallazgos: se identificaron limitaciones relevantes en el desempeño de *Guardrails* en español, lo que ocasionó respuestas inconsistentes y filtros poco precisos.
- Recomendaciones: reforzar el soporte multilingüe de *Guardrails* o evaluar alternativas más compatibles con el idioma español. Incorporar mensajes estandarizados para manejar adecuadamente consultas fuera de contexto o sin respuesta posible.

Conclusiones de la versión 1

La versión 1 representó un avance significativo respecto a la versión inicial, al mejorar la modularidad, el control de costos y la calidad de las respuestas. LanceDB optimizó el manejo de *embeddings* y DSPy facilitó una implementación más clara del *pipeline* RAG. También se incorporaron mecanismos de filtrado y evaluación automática. No obstante, persistieron limitaciones en la retención del contexto en conversaciones largas y en el rendimiento de NeMo *Guardrails* en español. Estos resultados evidenciaron la necesidad de fortalecer la continuidad conversacional y el control lingüístico. En conjunto, la versión 1 estableció una base técnica sólida para futuras mejoras funcionales.

4.4.3. Pruebas para la versión 2

Pruebas de interacción básica

En las pruebas de interacción básica, la versión 2 presentó mejoras significativas respecto a la anterior. Sin embargo, se observó un aumento en la latencia debido al almacenamiento de conversaciones completas en RAM, lo que afectó ligeramente los tiempos de respuesta en consultas extensas.

- Recomendaciones: optimizar el uso de RAM para reducir la latencia mediante un sistema de almacenamiento más eficiente o basado en la nube, como REDIS [11].

Pruebas de consultas con múltiples pasos

En interacciones complejas que incluyeron consultas encadenadas, el sistema presentó un rendimiento mixto. Aunque la memoria conversacional permitió generar respuestas más coherentes, en algunos casos el contexto no se interpretó correctamente, especialmente cuando las preguntas no hacían referencia explícita a elementos previos. Por ejemplo, ante la pregunta “¿Cuánto cuesta?” tras una consulta sobre un producto, el sistema no logró identificar adecuadamente el antecedente, lo que condujo a respuestas incorrectas.

- Hallazgos: la memoria conversacional mejoró la coherencia general, pero persistieron dificultades en la interpretación del contexto en preguntas encadenadas.
- Recomendaciones: incorporar un sistema de reformulación automática de preguntas para enriquecer el contexto y mejorar la precisión de las respuestas. Mejorar la detección de cambios temáticos sutiles o ambigüedades en la formulación de consultas.

Pruebas de consultas fuera de contexto

El sistema de *guardrails* personalizados en la versión 2 mostró mayor precisión en la detección y bloqueo de consultas fuera de contexto. Las preguntas irrelevantes o mal formuladas fueron redirigidas con mayor efectividad.

Conclusiones de la versión 2

La versión 2 mostró avances relevantes en coherencia, control contextual y filtrado de preguntas, especialmente en español. No obstante, persistieron limitaciones en la interpretación de referencias implícitas y en la latencia durante consultas extensas. Se recomienda optimizar el uso de memoria y reforzar los mecanismos de reformulación automática.

4.4.4. Pruebas para la versión 3

Pruebas de interacción básica

En las pruebas de interacción básica, el sistema respondió adecuadamente a preguntas simples y directas y mantuvo la coherencia y relevancia en las respuestas.

- Hallazgos: el sistema respondió con precisión en la mayoría de los casos gracias a la estructura RAG y al uso de contextos organizados mediante pares clave-valor. Se observó una mejora notable en la coherencia y contextualización de las respuestas, especialmente en consultas complejas.
- Recomendaciones: reforzar la cobertura de dominios poco frecuentes mediante la expansión del conjunto de datos contextuales. Incluir ejemplos que aborden productos nuevos o condiciones especiales.

Pruebas de consultas con múltiples pasos

Se evaluaron interacciones en las que la respuesta dependía de la acumulación de información a lo largo de varios turnos. Este tipo de prueba fue clave para validar la persistencia del contexto y la capacidad de recuperación de información distribuida.

- Hallazgos: el módulo de reformulación de preguntas mejoró la comprensión de consultas multietapa.

Pruebas de consultas fuera de contexto

Para las preguntas fuera de contexto, no se observaron cambios significativos en comparación con la versión 2. El sistema mantuvo *guardrails* personalizados para filtrar preguntas irrelevantes.

- Hallazgos: se logró trazabilidad completa de las interacciones, evaluación automática y una arquitectura modular consolidada. No obstante, se detectó un rendimiento inferior en categorías como quejas, posiblemente debido a la falta de contexto temático suficiente.
- Recomendaciones: ampliar la base contextual en áreas críticas como atención al cliente. Evaluar la implementación de mecanismos de autoajuste más frecuentes y rediseñar los *prompts* de reformulación para mejorar la recuperación semántica en casos límite.

Conclusiones de la versión 3

La integración de Langfuse, junto con DSPy y LanceDB, representó un avance significativo en términos de trazabilidad, evaluación y diseño modular del sistema. Se mejoró tanto la precisión como la eficiencia sin incrementar los costos operativos.

4.4.5. Comparación de versiones del chatbot según métricas de aplicación

La incorporación del sistema de bases de datos en la versión 2 permitió realizar pruebas retrospectivas sobre versiones anteriores y almacenar los resultados en dicha base con el propósito de analizarlos posteriormente. Con la inclusión de Langfuse en la versión final, se migró esta información desde la base de datos obsoleta hacia Langfuse, lo que habilitó la ejecución de agentes automáticos sobre todas las trazas correspondientes a las cuatro versiones. Esto posibilitó un análisis retrospectivo de la evolución del modelo a lo largo del proceso. En la tabla 4.1 se resumen las principales métricas de desempeño obtenidas durante el desarrollo del chatbot. Se incluyen el número de interacciones registradas, el costo asociado y el puntaje promedio de evaluación automática.

TABLA 4.1. Comparación detallada de métricas entre diferentes versiones del chatbot.

Versión	Trazas	Costo (USD)	Puntaje Promedio
0	96	0,0506	5,88
1	132	0,0436	7,78
2	54	0,0126	8,29
3	136	0,0353	6,33

A lo largo del desarrollo del chatbot, se observó una disminución progresiva del costo por interacción, que alcanzó su punto más bajo en la versión 2. Esta optimización económica no fue producto del azar, sino el resultado directo de decisiones técnicas orientadas a reducir la cantidad de tokens procesados por el modelo de lenguaje.

En la versión 0, el sistema utilizaba Pinecone como base vectorial y almacenaba fragmentos extensos de documentos que se incorporaban directamente al *prompt*. Este enfoque implicaba altos volúmenes de texto por consulta, lo que generaba un mayor consumo de tokens y, por ende, un costo más elevado por interacción (USD 0,0506 por 96 trazas).

A partir de la versión 1, consolidado en la versión 2, se reemplazó Pinecone por LanceDB, una base de datos vectorial local que permitió no solo reducir los costos asociados a infraestructura externa, sino también mejorar la precisión de recuperación mediante el control total del almacenamiento.

Sin embargo, la optimización más significativa se logró mediante la redefinición de la estructura de contexto. Se pasó de fragmentos textuales extensos a una arquitectura basada en pares clave-valor semánticos, donde cada clave correspondió a una intención o categoría de consulta y cada valor contuvo únicamente la información estrictamente necesaria. Este diseño minimizó el tamaño de los

contextos insertados en los *prompts* y redujo drásticamente el número de tokens utilizados por consulta.

Esta estrategia se reflejó en la métrica de la versión 2. Con solo 54 trazas se alcanzó un costo total de USD 0,0126 y un puntaje promedio de calidad de 8,29, el más alto entre todas las versiones. Aunque la versión 3 presentó un leve aumento en el costo (USD 0,0353), este incremento se debió a la mayor complejidad asociada al manejo de categorías y a la redefinición de las consultas, lo que generó un incremento en la cantidad de tokens procesados en comparación con la versión 2.

En resumen, la reducción del costo operativo del sistema se debió a la adopción de LanceDB como *vectorstore* local y la estructuración eficiente del conocimiento en pares clave-valor, lo que redujo sustancialmente el consumo de tokens sin comprometer la calidad de las respuestas.

Capítulo 5

Conclusiones

5.1. Conclusiones generales

Este trabajo abordó el diseño, desarrollo e implementación de un sistema modular de aprendizaje continuo para chatbots basados en Procesamiento de Lenguaje Natural (PLN). El objetivo fue ofrecer soluciones accesibles, escalables y sostenibles para pequeños emprendimientos. La arquitectura se basó en el enfoque RAG (por sus siglas en inglés, *Retrieval-Augmented Generation*), lo que permitió generar respuestas precisas y contextualizadas, con capacidad de mejora continua sin necesidad de reentrenamiento.

5.1.1. Cumplimiento de objetivos y planificación

Los objetivos planteados fueron alcanzados en su totalidad. Se desarrolló un *pipeline* funcional, de bajo costo y fácil implementación, que incorpora mecanismos de evaluación automática y trazabilidad.

En cuanto a la planificación, se respetaron los recursos estimados (horas de trabajo y presupuesto de cómputo en la nube), aunque los plazos inicialmente definidos no se cumplieron.

5.1.2. Gestión de riesgos y adaptaciones

Durante el desarrollo se identificaron limitaciones en la retroalimentación basada en historiales. Este riesgo fue abordado mediante la incorporación de Langfuse, una herramienta que permitió mejorar la trazabilidad de las interacciones y fortalecer la evaluación automática del sistema.

5.1.3. Resultados, aprendizajes y aportes

El sistema evidenció un rendimiento sólido en distintos escenarios, que incluye consultas de múltiples pasos o fuera de contexto. Se identificaron mejoras significativas en la coherencia y relevancia de las respuestas a lo largo de las distintas versiones del prototipo. Las estrategias de reformulación de preguntas, clasificación temática y recuperación semántica resultaron determinantes para alcanzar estos resultados sin necesidad de reentrenar el modelo.

Desde una perspectiva formativa, el trabajo integró múltiples conceptos propios de la Especialización en Inteligencia Artificial. Entre ellos destacan el diseño de arquitecturas modulares, la evaluación automática, el procesamiento semántico, los vectores de *embeddings*, los sistemas conversacionales y el despliegue en la nube. Estas herramientas permitieron desarrollar una solución técnicamente sólida

y contribuyeron a una comprensión profunda de los desafíos asociados con la implementación de aplicaciones basadas en PLN en entornos reales.

Los principales aportes del trabajo incluyen:

- Una arquitectura modular basada en DSPy y LanceDB, adaptable y de bajo mantenimiento.
- Un sistema integral de evaluación conversacional, con métricas automáticas, análisis semántico y agrupamiento temático.
- Un entorno desplegado en *Azure Container Apps*, listo para su utilización en pruebas reales sin depender de infraestructura externa costosa.

5.2. Próximos pasos

Una evolución natural del sistema consiste en integrar el módulo RAG como un agente especializado dentro de un chatbot multiagente. Por ejemplo, en un entorno gastronómico podrían definirse distintos agentes colaborativos:

- Un agente de atención al cliente, encargado de responder preguntas frecuentes sobre horarios, menú o ubicación.
- Un agente de pedidos, encargado de registrar solicitudes de clientes y almacenarlas en una base de datos interna.
- Un agente CRM (por sus siglas en inglés, *Customer Relationship Management*), conectado a sistemas externos para la gestión de datos de clientes y su historial de interacciones.
- Un agente RAG, responsable de atender consultas complejas o basadas en documentación, que utilice la arquitectura desarrollada en este trabajo.

Este tipo de arquitectura distribuida y colaborativa permitiría construir asistentes conversacionales más flexibles, escalables y adaptados a distintos dominios, con una lógica clara de mantenimiento y evolución continua.

5.3. Síntesis final

El desarrollo realizado establece las bases para una nueva generación de chatbots inteligentes, modulares y autoevaluables. La arquitectura propuesta demuestra que es posible diseñar soluciones conversacionales robustas, sin necesidad de reentrenamiento, con bajo costo operativo y aplicabilidad directa en contextos reales. Este trabajo representa una contribución concreta a la implementación práctica de tecnologías de PLN en entornos accesibles para pequeñas organizaciones.

Bibliografía

- [1] IBM. *Natural Language Processing (PLN)*. <https://www.ibm.com/mx-es/topics/natural-language-processing>. Accedido el 17 de septiembre de 2024. 2023.
- [2] Cobus Greyling. *RAG vs Fine-Tuning*. <https://cobusgreyling.medium.com/rag-fine-tuning-e541512e9601>. Accedido el 20 de septiembre de 2024. 2023.
- [3] Gustavo Espindola. *¿Que son los embeddings y como se utilizan en la inteligencia artificial con python?* <https://gustavo-espindola.medium.com>. Accedido durante septiembre de 2024. 2023.
- [4] GeeksforGeeks. *What is an AI Prompt?* <https://www.geeksforgeeks.org/what-is-an-ai-prompt/>. 2021.
- [5] Google. *¿Qué es una base de datos vectorial?* <https://cloud.google.com/discover/what-is-a-vector-database?hl=es>.
- [6] Nvidia. *NVIDIA NeMo Guardrails para desarrolladores*. <https://developer.nvidia.com/nemo-guardrails?>.
- [7] Microsoft. *What is software as a service (SaaS)?* <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-saas>.
- [8] Tahir. *What is Ollama: running large language models locally*. <https://medium.com/@tahirbalarabe2/what-is-ollama-running-large-language-models-locally-e917ca40defe>. 2024.
- [9] FastApi. *FastApi: Docs*. <https://fastapi.tiangolo.com/>.
- [10] HDBSCAN: Docs. *How HDBSCAN Works*. https://hdbscan.readthedocs.io/en/latest/how_hdbscan_works.html.
- [11] IBM. *¿Qué es Redis?* <https://www.ibm.com/mx-es/topics/redis>.